



FACULTY OF INFORMATION TECHNOLOGY

INTRO O/S

Lab Instructions 9.3 – Banker Algorithm.

In the system there are many types of resources, each resource has many entities. Processes will declare the number of entities they need to execute and complete. In the role of resource distribution and management, the operating system needs to evaluate the security of the system to have appropriate timing and allocation plans.

Target	Related theory	Resources
Safety Algorithm	Ch8.6.3 Banker's Algorithm	https://github.com/Trantin84/LAB_IntroOS (example source code). Use Ubuntu 16/18 image
Resource Request Algorithm	Ch8.6.3.1 Safety Algorithm	
Banker's algorithm	Ch8.6.3.2 Resource-Request Algorithm	

Ask students: Understand and execute the provided code snippets. Applied to problems of finding Safe strings.

Student reviews: Questions and answers about theoretical issues. Practical skills. Exercise.

Submission requirements: .c source code files and .out executable files of the Practice Session Requirements and Final Tutorial Exercises within the time allowed by the instructor.

Preferences

[1] Abraham Silberschatz, Peter B. Galvin, Greg Gagne, [2018], Operating System Concepts, 10th edition, John Wiley & Sons, New Jersey.

Programming Problems of Chapter 8.

[2] Vikash Kumar, [2020], Banker's Algorithm in Operating System, Geeksforgeeks.org.

Access <https://www.geeksforgeeks.org/bankers-algorithm-in-operating-system-2/>

Requirements 1: Find the safety chain of the System given in Example 8.6.3.3 (book [1] page 336) (also in Lecture Slide Ch8 slide 31).

Instruct: Download file LAB_IntroOS/LAB_9/task3_1_SafetyAlgorithm.c, compile and execute. Note that it may be necessary to reset the matrix values to match the problem parameters.

```
$ gcc -o banker.out task3_1_SafetyAlgorithm.c
$ ./banker.out 3 3 2
$SAFE Sequence: P1 -> P3 -> P4 -> P0 -> P2.

$ ./banker.out 3 2 2
$ The system is UNSAFE.
```

Requirement 2: (Students self-study) (Resource-Request Algorithm) Function implementation

int request_resources(int customer_num, int request[]);

* Parameter: customer_num is the process index requesting more resources and the request[] array contains the number of entities that the customer_num process wants to have immediately.

* Returns: if allocation fails, -1 is returned; otherwise 0 is returned.

Instruct:

- Theory: book [1] page 336 and Lecture Slide Ch8 slide 30.

- Pseudocode: at LAB_IntroOS/LAB_9/task3_2_Resource_Request_Pseudocode.c

Requirement 3: (Students self-study) Function realization

void release_resources(int customer_num, int request[]);

* Parameters: customer_num is the process index returning the resource and the request[] array contains the number of entities that the customer_num process must return immediately.

* Returns: no.

Application: in some Congestion, or Resource Exhaustion states; The System will reclaim the entities that one or several processes are holding (In fact, the System will kill the process completely). This increases available resources and can help the system continue to execute.

Exercise

1. Add code to Requirement 1 to read the file and input the maximum and allocation matrices from the file.

Suggest: sample files are given task3_maximum.txt and task3_allocation.txt. Similar file reading function in LAB 4.3 – Requirement 2 (source code LAB_IntroOS/LAB_4/task3_2.c), in which, the file reading function can be used for 2 threads to read 2 files at the same time.

2. The parameter passed in the call is the available vector. If this parameter is "the total number of initially available system resources", then how is the available vector calculated? Please modify the code in Requirement 1.

Suggest: available is the number of resource entities remaining after the system has allocated them to processes. And the total number of allocated resource entities of a type of process is the sum of the elements in the column representing that resource in the allocation matrix.

The call in Request 1 is now equivalent to:

```
$ ./banker.out 10 5 7
$SAFE Sequence: P1 -> P3 -> P4 -> P0 -> P2.
```